

《密码学导论》课程大作业

作品设计报告

作品类别： 软件设计 硬件制作 工程实践

作品题目： 随机数的统计分布测试与蒙特卡罗法求 π

团队名称： 个人

团队人员： PB24331906 张鹏飞

日期： 2026 年 6 月 14 日

基本信息表

作品题目	随机数的统计分布测试与蒙特卡罗法求 π
作品内容摘要	本项目设计并实现了一套针对 C 标准库 <code>rand()%N</code> 随机数的统计分布测试工具，实现了卡方、K-S、序列、游程、自相关、累加和共六种经典检验；同时采用蒙特卡罗方法估算 π 以暴露随机数的二维不均匀性。进一步，依据 NIST SP800-22 标准提取最低有效位 (LSB) 进行比特级随机性检测，从宏观统计和微观结构两个层面全面评价 <code>rand()%N</code> 的伪随机质量。实验结果表明， <code>rand()%N</code> 能满足简单的一维均匀性，但在二维模拟与密码学比特检测中明显失败，为实际应用中选择安全随机源提供了依据。
关键词	随机数检验；蒙特卡罗；NIST SP800-22； <code>rand()</code> ；均匀分布
团队成员	见表后附

序号	姓名	学号	任务分工
1	张鹏飞	PB24331906	整体架构设计、核心 C 程序编写、整数层面检验实现
2			蒙特卡罗求 π 、均匀/正态分布生成、结果分析
3			NIST 比特级测试、Python 脚本开发、报告撰写

1 作品功能与性能说明

本作品包含两大功能模块：

1. 随机数生成与多维统计检验

- 生成 $[0, N - 1]$ 范围内的随机整数（默认 $N = 100$ ，样本量 100 000）。
- 自动统计每个数值的出现频率与重复间隔分布。
- 内置六种独立性/均匀性检验：卡方拟合优度检验、K-S 检验、序列（二元组）检验、游程检验、自相关检验、累加和检验，并输出统计量及通过/未通过判断。
- 提供改进的均匀分布（浮点映射法）和正态分布（Box-Muller 变换）随机数生成函数。
- 采用蒙特卡罗法估算圆周率 π ，直观展示低质量随机数对模拟精度的危害。

2. NIST SP800-22 比特级随机性检测

- 将整数序列转换为平衡的比特流（提取 LSB），满足 NIST 测试对 $p = 0.5$ 的要求。
- 支持频率检验、块内频率检验、游程检验、最长游程检验、近似熵检验、累加和检验、序列检验共 7 项核心测试。
- 输出各测试的 p -value 及通过/未通过结论，可精确识别 `rand()` 低位比特的规律性。

性能指标：

- 在普通 PC 上生成 10 万随机数并完成全部整数检验耗时 < 1 秒。
- 蒙特卡罗 100 万点估算 π 耗时约 2-3 秒。
- NIST 100 万比特测试（含扩展）耗时 < 5 秒（Python + numpy）。

2 设计与实现方案

2.1 实现原理

2.1.1 系统总体流程

1. 使用 `rand() % 100` 生成 100 000 个整数序列。
2. 统计每个值的频率、间隔分布。
3. 执行六种经典统计检验（卡方、K-S、序列、游程、自相关、累加和）。
4. 使用相同随机源进行蒙特卡罗求 π 实验，记录偏差。
5. 导出包含整数的二进制文件（每数 1 字节），供后续比特分析。
6. 提取最低有效位（LSB），平铺至 10^6 比特，运行 NIST 核心测试。

2.1.2 核心算法说明

- **改进均匀整数**：`(int)((rand() / (RAND_MAX + 1.0)) * N)`，避免低位偏置。
- **正态分布生成**：采用 Box-Muller 变换，

$$z = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad U_1, U_2 \sim U(0, 1)$$

- **六种检验算法**（均在 C 程序中实现）：

– 卡方检验： $\chi^2 = \sum (O_i - E_i)^2 / E_i$ ，自由度 $N - 1$ 。

- K-S 检验: $D = \max |F_n(x) - F(x)|$, 比较经验与理论分布。
 - 序列检验: 对二元组 (x_j, x_{j+1}) 进行卡方检验。
 - 游程检验: 中位数二值化后统计游程数, 计算 Z 值。
 - 自相关检验: 计算滞后 $d = 1$ 的皮尔逊相关系数。
 - 累加和检验: 转换为 ± 1 后计算前向/后向累积和的最大绝对值。
- **NIST 比特级检验:** 提取 LSB 后, 运行各项测试, 使用 `erfc` 和 `gammaincc` (或 Wilson-Hilferty 近似) 计算 p -value。

2.1.3 关键代码片段

```
// 蒙特卡罗求
for (int i = 0; i < num_points; i++) {
    double x = (rand() % 10000) / 1000.0; // [0,10)
    double y = (rand() % 10000) / 1000.0;
    if (x*x + y*y <= 100.0) inside++;
}
pi = 4.0 * inside / num_points;
```

```
# 提取 LSB 并扩展至 1M bits
data = np.fromfile("rand_sequence_byte.bin", dtype=np.uint8)
lsb = data % 2
bits = np.tile(lsb, 10) # 100k -> 1M bits
results = run_all_battery(bits, SP800_22R1A_BATTERY)
```

2.2 参考文献

1. NIST Special Publication 800-22 Rev. 1a, *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, 2010.
2. Knuth D. E., *The Art of Computer Programming*, Vol.2, Seminumerical Algorithms, 3rd ed., Addison-Wesley, 1998.
3. Box G. E. P., Muller M. E., *A Note on the Generation of Random Normal Deviates*, Ann. Math. Statist. 29, 1958.
4. 中华人民共和国密码行业标准 GM/T 0005-2021 《随机性检测规范》。

2.3 运行结果

2.3.1 频率统计 (前 20 个整数)

Value	Count	Prob	TheoProb
0	994	0.00994	0.01000
1	1020	0.01020	0.01000
2	975	0.00975	0.01000
3	987	0.00987	0.01000
4	975	0.00975	0.01000

... (总计 100 个数值, 频率在 0.0095~0.0107 之间)

2.3.2 重复间隔分布

总间隔数: 99 900, 最大间隔: 1406。直方图显示间隔近似指数衰减, 无异常聚集。

2.3.3 六种检验结果

表 1: 整数层面六种检验结果

检验项目	统计量	结论
卡方 (χ^2)	95.24	通过
K-S D	0.01180	临界 (Δ)
序列 (pairs χ^2)	9917.60	通过
游程 Z	-1.199	通过
自相关 (lag=1)	-0.0005	通过
CUSUM max	278.00	通过

2.3.4 蒙特卡罗求 π

- 投点次数: 1 000 000
- 估计值: **3.271228**
- 绝对误差: **0.1296** (理论标准误 ≈ 0.0005)

2.3.5 NIST SP800-22 比特级测试 (LSB, 1 000 000 比特)

2.4 技术指标

- 可测序列长度: 整数 $10^3 \sim 10^7$, 比特 $10^5 \sim 10^7$ 。

表 2: NIST SP800-22 比特级测试结果

测试项	p-value	结果
Frequency (Monobit)	0.214975	PASSED
Block Frequency (M=128)	0.083147	PASSED
Runs	0.901681	PASSED
Longest Run of Ones	0.747497	PASSED
Cumulative Sums	0.106527	PASSED
Serial (m=3)	0.009885	FAILED
Approximate Entropy	0.000000	FAILED

- 检验覆盖度：整数层面 6 种经典检验 + 比特层面 7 项 NIST 核心测试（可扩展至 15 项）。
- 环境兼容性：C 代码跨平台（GCC / MSVC），Python 依赖仅 numpy。

3 系统测试与结果

3.1 测试方案

1. **整数层面测试**：生成 100 000 个 `rand()%100` 整数，运行六种检验，记录统计量与判断标准。
2. **蒙特卡罗 π 测试**：生成 1 000 000 个 `rand()%10000 / 1000.0` 坐标点，统计圆内比例并计算误差。
3. **NIST 比特测试**：从 100 000 个整数提取 LSB，平铺至 10^6 比特，运行 7 项核心测试，显著性水平 $\alpha = 0.01$ 。
4. **对比测试**：生成改进的均匀分布（浮点映射法）和正态分布（Box-Muller）样本，目视检查合理性。

3.2 功能测试

- 频率统计功能正常，各数值概率分布在理论值 0.01 附近小幅度波动。
- 间隔分布检测正常，输出直方图符合几何分布特征。
- 六种检验均能成功计算并输出对应统计量，无运行错误。
- 蒙特卡罗模块能正确估算 π （尽管数值偏差较大）。
- NIST 模块正确执行 7 项测试，成功识别出不符合随机性假设的项。

3.3 性能测试

- C 程序生成 10 万整数并完成全套检验、蒙特卡罗 100 万点，内存占用 < 10 MB，CPU 时间 < 3 秒。
- Python NIST 测试处理 100 万比特，内存约 20 MB，CPU 时间约 2-4 秒（含 numpy 加速）。

3.4 测试数据与结果

整数检验：如表 ?? 所示，卡方、序列、游程、自相关、CUSUM 均顺利通过 ($p > 0.01$)，仅 K-S 出现临界值。这表明一维均匀性基本成立。

蒙特卡罗 π ：误差高达 0.1296，远超统计预期，说明 `rand()%N` 生成的二维坐标并非均匀分布，存在明显的聚集或排斥模式。

NIST 比特检验：结果如表 ?? 所示。整体 0/1 频率较均衡，因此频率类检验通过。未通过的序列检验说明相邻比特的短程依赖性显著；近似熵检验 $p = 0$ 则表明序列复杂度远低于真随机比特流，可压缩性高。这正是 `rand()` 低比特周期短、规律性强的典型表现。

4 应用前景

本作品可广泛用于：

- **教学与科研：**为密码学、随机数课程提供直观的演示工具，帮助学生理解伪随机数的统计特性。
- **随机数发生器测评：**可在开发密码协议或模拟软件时，快速评估自研或第三方随机源的均匀性与独立性。
- **质量保障：**在金融蒙特卡罗模拟、密码抽签等场景中，可对使用的随机数进行预检，避免因低质随机源导致系统偏差或安全漏洞。
- **工具扩展：**通过增加 NIST 全项 15 测试、图形化界面，可形成商业或开源随机数检测套件。

5 结论

本作品通过 C 语言和 Python 联合实现了对 `rand()%N` 随机数的多维度测评。

- **整数层面：**传统六种检验基本通过，给人一种“均匀独立”的假象。
- **蒙特卡罗模拟：** π 估计严重偏离真值，暴露了序列在二维空间的非均匀性。

- **NIST 比特层检验：**LSB 流无法通过序列检验和近似熵检验，证实 `rand()` 的低位比特存在强规律性和低复杂度，不满足密码学随机性要求。

综上，C 标准库的 `rand()%N` 仅适用于对随机质量要求不高的场合；在密码学、安全协议或高精度模拟中，必须替换为密码学安全的随机数生成器。本作品为随机数质量的科学评估提供了一套高效、可信的检测方案。